Kron-Reduced vs. Full Graph Laplacians for Water-Quality Regression on the Colorado River Network

Andrew Chincea
TDA Lab

November 2026

Abstract

The graph-Laplacian water-quality models for the Colorado basin use a Kron (Schur-complement) reduction of the 67,401-node river network onto the 26 nodes with water-quality (WQ) stations. We test whether this reduction changes the model. For five MERIT-based edge weightings, on both the uncompressed and the compressed (67-node) graph, the Kron and full Laplacians agree to within 10^{-8} on every quantity the reduction should preserve. Re-fitting the nested leave-one-node-out (LOO) regression with the full Laplacian changes R^2 by at most 10^{-10} and selects identical hyperparameters in every fold. Kron reduction is therefore exact, and about $150\times$ faster. We also correct several problems in the original analysis. The most consequential is that feature sets chosen by looking at outer-LOO results overstated performance: by about 0.13 in R^2 for specific conductance and 0.02–0.08 for DO, pH and turbidity. With selection done inside the inner loop, the graph term clearly helps for pH and specific conductance, while seasonal and MERIT predictors matter much more for water temperature, DO and turbidity. In a full rerun of the eight-model benchmark on identical folds, the graph model ranks first for specific conductance and pH, within 0.005 of the best for water temperature and DO, and seventh of eight for turbidity. When every model also selects its features inside the inner loop, the graph model still wins clearly for specific conductance (0.207) and pH (0.130). It ties for water temperature, falls to fifth for DO and rises to 0.106 for turbidity, where gradient boosting leads with 0.391. Finally, removing edge weights changes almost nothing on the original graph. On the compressed graph, however, an unweighted Laplacian (one step per river reach), chosen inside the inner loop, raises the graph model's specific conductance R^2 from 0.207 to 0.385.

1 Setup

Data. Hourly USGS measurements at 28 stations, mapped to 26 river-network nodes. Depending on the target, 17–24 nodes have usable data (DO 17, water temperature 24, pH 19, specific conductance 24, turbidity 17). The network is the largest connected component of the MERIT-derived Colorado graph: 67,401 nodes and 67,400 edges, each edge about 70–120 m long.

Model. For node i and hour t, the prediction is $\hat{y}_{it} = \beta_0 + z_{it}^\top \beta + g_i$, where z_{it} are standardized predictors and g_i is a node offset. The fit minimizes

$$\sum_{i \in \text{train}} \frac{1}{n_i} \sum_t \left(y_{it} - \beta_0 - z_{it}^\top \beta - g_i\right)^2 \;+\; \lambda_\beta \sum_{k \geq 1} \beta_k^2 \;+\; \lambda\, g^\top \tilde{L}\, g, \qquad \text{s.t. } \textstyle\sum_{i \in \text{WQ}} g_i = 0, \tag{1}$$

with $g^\top L g = \sum_{(i,j)} w_{ij}(g_i - g_j)^2$ and $\tilde{L} = L/s$. The scale s is the mean positive diagonal of the Kron matrix. The weight $1/n_i$ makes each station count equally. A held-out node has no data term,

so its offset is set by the smoothness penalty alone. The hyperparameters λ and λ_β (and, where stated, the edge weighting) are tuned by an inner LOO. Performance is the node-weighted R^2 of the outer-LOO predictions.

Edge weightings. $w_{ij} = \exp(\text{score}_{ij})/d_{ij}$, where d_{ij} is the edge length in km and the score mixes z-scored log upstream area, log width and log slope. There are five schemes: `distance_only`, `area`, `width`, `slope` and `area_width`.

Kron reduction. Partitioning nodes into WQ nodes B and the rest I,

$$L_{\text{kron}} = L_{BB} - L_{BI}L_{II}^{-1}L_{IB} \in \mathbb{R}^{26\times 26}. \tag{2}$$

The non-WQ nodes have no data term in (1), so minimizing over their g in closed form leaves exactly $g_B^\top L_{\text{kron}} g_B$. The two formulations should therefore be the same model. This report checks that numerically.

2 Are the Laplacians the same?

The Kron and full matrices have different sizes, so we compare what Kron reduction must preserve:

- **Effective resistance** between every pair of WQ nodes, computed on the full Laplacian by sparse solves and on L_{kron}.
- **Dirichlet energy:** for random WQ values x_B with harmonic extension x, $x^\top Lx$ should equal $x_B^\top L_{\text{kron}} x_B$.
- **Compression:** Kron(uncompressed) vs Kron(compressed), compared directly.

As a reference for what a *wrong* reduction looks like, we also compare against the sub-matrix L_{BB}, i.e. simply deleting the non-WQ nodes.

Table 1: Largest relative errors, uncompressed graph. All values are at rounding-error level, except the deliberately wrong L_{BB} reference.

Weighting	Eff. resistance	Energy	Kron unc. vs comp.	Naive L_{BB} (reference)
`distance_only`	1.9×10^{-9}	2.4×10^{-12}	6.7×10^{-13}	205
`area`	4.7×10^{-9}	1.5×10^{-12}	5.1×10^{-13}	368
`width`	4.1×10^{-10}	3.1×10^{-12}	6.4×10^{-13}	253
`slope`	7.3×10^{-9}	8.8×10^{-12}	2.6×10^{-12}	257
`area_width`	6.0×10^{-9}	2.6×10^{-12}	8.1×10^{-13}	359

On the compressed graph, the resistance and energy errors are below 1.5×10^{-12} for every weighting. Compression is also exact. Each compressed edge replaces a chain of resistors in series, and series resistances add, so the total resistance between stations is unchanged.

3 Does Kron change the regression?

The nested LOO was re-run with four Laplacians, using exactly the same solver code:

Variant	Laplacian in (1)	Size
`kron_uncompressed`	Kron of the 67k graph (original method)	26×26
`kron_compressed`	Kron of the 67-node graph	26×26
`full_uncompressed`	full Laplacian, g on every node	$67{,}401 \times 67{,}401$
`full_compressed`	full Laplacian, g on every node	67×67

The full variants never form a Schur complement. They solve the complete (β, g) system by block elimination with a sparse factorization. Each full Laplacian is divided by the same s as its Kron reduction, so λ has the same meaning in all four.

Table 2: Outer-LOO R^2, chemistry predictors only. "Graph, $1/d$" is `distance_only`. The last column is the largest change in R^2 between `kron_uncompressed` and any other variant, over both graph modes.

Target	Linear	Graph, $1/d$	Graph, tuned weighting	Max $\lvert\Delta R^2\rvert$
DO concentration	0.6208	0.6135	0.6065	3.4×10^{-11}
Water temperature	−0.0556	−0.0346	−0.0575	9.9×10^{-11}
pH	−0.0352	0.0742	0.0742	1.6×10^{-11}
Specific conductance	−0.0891	0.0907	0.1062	2.2×10^{-11}
Turbidity	0.0228	−0.0486	−0.0360	4.4×10^{-11}

- **Identical models.** All four variants chose the same (weighting, λ, λ_β) in 100% of outer folds for every target. R^2 differs by at most 10^{-10}.
- **Reproduction.** `kron_uncompressed` reproduces the original notebook's saved results (`full_graph_nested_loo_results.csv` and the `Linear + Laplacian` rows of `all_models_node_loo_results.csv`) to within 1.3×10^{-12}.
- **Cost.** Kron took 0.1 s per outer fold vs 15.2 s for the full 67k Laplacian (26 minutes in total), about 150× faster.

When the graph term helps. With chemistry predictors only, the graph term improves pH (+0.11) and specific conductance (+0.18 to +0.20). Both are plausibly controlled by geology and therefore spatially structured. It is neutral or slightly harmful for DO, water temperature and turbidity. Tuning the MERIT weighting instead of using $1/d$ changes R^2 by at most 0.02. The weighting chosen is also unstable from fold to fold for DO (`width` in 13 of 17 folds) and for water temperature (`area` 15, `slope` 5, `area_width` 4 of 24). For pH, `distance_only` was chosen in all 19 folds.

4 Corrections to the original analysis

4.1 pH transforms with the scaled Laplacian

Predictors are specific conductance plus $K = 1$ Fourier terms.

Raw and z-scored pH give identical results, as expected for an affine transform. Fitting H^+ is worse, and no H^+ predictions had to be clipped. Raw pH should be kept.

Table 3: Problems found in `largest_uncompressed_laplacian.ipynb` and how they were handled.

Problem	Fix
pH-transform section used the *unscaled* Kron Laplacian (λ about $21\times$ weaker)	scaled Laplacian; unscaled run alongside for comparison
Static feature sets and Fourier K chosen from outer-LOO results, then re-scored on the same folds	selection moved inside the inner LOO
"Discharge" edge feature is a linear function of log UPA, so it duplicates `area`; its fit also used held-out stations	discharge removed
HAND is 0 at all 26 WQ nodes (river pixels)	HAND removed from the predictors
Comparison tables used hard-coded R^2 values from earlier runs	all tables computed from the run

Table 4: pH nested-LOO R^2. The scaling fix changes graph R^2 by only -0.002, because tuning λ over a wide grid absorbs the constant factor.

Target transform	Linear	Graph, scaled (fixed)	Graph, unscaled (original)	Change
raw pH	0.0163	0.1300	0.1315	-0.0016
z-scored pH	0.0163	0.1300	0.1315	-0.0016
$H^+ = 10^{7-\mathrm{pH}}$	-0.0662	0.0930	0.0954	-0.0024

4.2 Feature selection inside the nested LOO

In each outer fold, the feature set was chosen by inner-LOO MSE alone, separately for the linear and graph models (graph: Kron Laplacians, with weighting, λ and λ_β tuned). The 30 candidates are chemistry plus Fourier $K \in \{1, \ldots, 5\}$, combined with one of six static options: none, one of elevation, log UPA, log width or local slope, or all four. The same run gives each candidate's own nested-LOO R^2. Taking the best of those reproduces what the original analysis effectively reported, so the difference measures how much it was overstated.

- **Water temperature.** A large and genuine gain, from 0.68 to 0.86, with almost no overstatement. All four MERIT features were chosen in every fold. With them included, the graph term adds nothing, because the node-level features already explain the offsets the Laplacian was capturing.
- **DO.** A modest genuine gain, to 0.70–0.71, driven by $K = 2$–4 seasonal terms with log UPA (graph, 13 of 17 folds) or elevation (linear, 14 of 17 folds). The graph is slightly worse than linear.
- **Specific conductance.** The largest overstatement. Choosing local slope after seeing outer results inflated graph R^2 from a fair 0.14 to 0.27, and the fair value is no better than the plain $K = 1$ baseline (0.146). The graph term is still the only model with positive R^2. Local slope was chosen in 22 of 24 folds, so it may be real, but it does not improve held-out accuracy.
- **pH.** The graph term helps (0.10 vs -0.06 for linear), but R^2 stays low and the chosen features vary from fold to fold.

Table 5: Outer-LOO R^2 with seasonal and MERIT predictors. "Biased" is the best candidate chosen after seeing outer results. "Fair" is the feature set chosen inside the inner loop, and is the number to report. "Baseline" is $K = 1$ with no static features.

Target	Model	Baseline	Biased	**Fair**	Overstated
Water temperature	linear	0.6781	0.8599	0.8573	0.0026
	graph	0.7394	0.8591	0.8568	0.0024
DO concentration	linear	0.6908	0.7260	0.7109	0.0151
	graph	0.6831	0.7220	0.7009	0.0211
pH	linear	0.0163	0.0268	−0.0553	0.0821
	graph	0.1300	0.1439	0.0977	0.0461
Specific conductance	linear	−0.0793	−0.0564	−0.1690	0.1126
	graph	0.1462	0.2674	0.1403	0.1271
Turbidity	linear	0.0554	0.1954	0.1443	0.0511
	graph	−0.0084	0.1064	0.0740	0.0324

- **Turbidity.** Linear with $K = 5$ seasonal terms and no static features is best (0.14). The graph term hurts.

5 Reconciliation with the Oct 29 and Nov 5 meeting notes

The model and theory in the meeting notes agree with this report. That includes the objective (1), the $1/n_i$ weighting, the Kron/Schur derivation, preservation of effective resistance, and the compression result. The reported Graph-Laplacian R^2 values differ. Every one of them was copied from an output cell of `largest_uncompressed_laplacian.ipynb`. Table 6 traces each one and gives two corrected values:

- **Benchmark refit:** the graph model fitted inside the updated `water_quality_model_benchmark.py`, on exactly the same rows, predictors and folds as the conventional models.
- **Fair:** the value from Table 5, with features selected inside the inner loop.

Table 6: Graph-Laplacian R^2 as reported in the notes, where each value came from, and the corrected values.

Target	Notes	Source in original notebook	Benchmark refit	Fair
Water temperature	0.8628	cell 118: reduced static set	0.8628	0.8568
DO concentration	0.7261	cell 131: DO base set, K tuned	0.7306	0.7009
Specific conductance	0.2525	cell 143: SC base set, K_{year} tuned	0.2569	0.1403
pH	0.1316	cell 67: unscaled Laplacian	0.1300	0.0977
Turbidity	0.0145	cell 129: other features/rows	0.0264	0.0740

Why the numbers differ.

1. **Post-hoc feature selection.** The Oct 29 notes state that "the outer held-out node is never used for tuning" (§9.1). That holds for λ, λ_β and K, but not for the static feature sets (`REDUCED_STATIC`, `DO_BASE_FEATURES`, `COND_BASE_FEATURES`). These were chosen by comparing

outer-LOO R^2 across candidate sets. This accounts for most of the gap between the notes and the fair values, above all for specific conductance ($0.25 \rightarrow 0.14$).

2. **The Nov 5 graph rows were not produced by the benchmark.** The original `water_quality_model_benchmark.py` never fitted the graph model. It read its rows from a `graph_laplacian_metrics.csv`, which does not exist, so the values in the notes were entered by hand.

3. **Turbidity was compared on different data.** The graph row was labelled 90,843 rows, but its value (0.0145, RMSE 561.81) comes from a model using specific conductance + pH + K=1 + slope on 88,095 rows. The conventional turbidity models use only slope + K=1 (`TURB_FEATURES`). On those rows and predictors, the graph model scores 0.026.

4. **Fixed vs tuned K.** For DO and specific conductance, the notes' graph values tuned K inside the loop, while the conventional models used fixed K (K=2; K_{day}=1, K_{year}=4). Refitting the graph model with the same fixed K changes R^2 by less than 0.005.

5. **pH scaling.** The notes' 0.1316 used the unscaled Laplacian. The scaled value is 0.1300, a negligible difference.

Updated benchmark. `water_quality_model_benchmark.py` (and the matching notebook) now fits the Graph-Laplacian model inside the same nested LOO as the seven conventional models. It uses the five saved Kron Laplacians, with weighting, λ and λ_β tuned by the same inner-LOO criterion. Fitting the graph model takes about 20 s for all five targets.

Full benchmark rerun. The complete benchmark (all eight models, all five targets) was rerun on the lab server with the updated script. The results are saved in `results/benchmark_results/`. Table 7 gives every model's R^2. Table 8 compares the graph model with the best conventional model and with the value in the notes. The graph values from the full run match the graph-only run exactly. The conventional models' values are unchanged from the Nov 5 table.

Table 7: Full benchmark rerun: node-weighted nested-LOO R^2 for every model on identical rows, predictors and folds. Bold marks the best model for each target. Stations and rows: water temperature 24 / 144,980; DO 17 / 114,607; specific conductance 24 / 144,980; pH 19 / 121,011; turbidity 17 / 90,843.

Model	Water temp.	DO	Spec. cond.	pH	Turbidity
Graph-Laplacian	0.8628	0.7306	**0.2569**	**0.1300**	0.0264
OLS	0.8634	0.7305	−0.1242	0.0116	0.0476
Ridge	**0.8640**	0.7252	−0.0997	0.0130	0.0393
Elastic Net	0.8635	0.7002	−0.1163	0.0226	0.0360
Spline GAM	0.8497	0.6458	0.0307	0.0023	0.1023
Random Forest	0.7541	0.6933	−0.0852	−0.0231	0.1003
Gradient Boosting	0.8221	**0.7358**	0.0011	0.0035	**0.3069**
Dummy	−0.0327	−0.0286	−0.0623	−0.0301	−0.0271
Graph rank (of 8)	4	2	1	1	7

The rankings in the Nov 5 notes are unchanged. The graph model is best for specific conductance (by 0.23 over the next model) and pH (by 0.11). It is in a near tie with the linear models for water temperature: four models fall within 0.0012 of each other, well inside the noise. It is second for DO,

Table 8: Best conventional model vs the Graph-Laplacian model from the full rerun, and the graph value reported in the notes. The feature sets are those in `TARGET_CONFIGS`, which were chosen post hoc, so every model's R^2 here is somewhat optimistic.

Target	Best conventional	R^2	Graph-Laplacian	Notes (graph)
Water temperature	Ridge	0.8640	0.8628	0.8628
DO concentration	Gradient Boosting	0.7358	0.7306	0.7261
Specific conductance	Spline GAM	0.0307	0.2569	0.2525
pH	Elastic Net	0.0226	0.1300	0.1316
Turbidity	Gradient Boosting	0.3069	0.0264	0.0145

0.005 behind gradient boosting and level with OLS. For turbidity it ranks seventh, ahead only of the dummy model, while gradient boosting reaches 0.31. Turbidity seems to depend on nonlinear effects that neither the linear models nor the graph model capture. The RMSE ranking agrees: the graph model has the lowest RMSE for specific conductance (382 vs at least 437 µS/cm) and for pH (0.221 vs at least 0.234). What changes is the size of the advantage once feature selection is done fairly (Table 5). For specific conductance, the graph model's fair R^2 is about 0.14, not 0.25. The conventional models also need their feature sets selected inside the loop before their numbers can be compared fairly. Section 6 does this.

6 Benchmark with inner-loop feature selection

Method. `water_quality_model_benchmark.py` now selects each model's feature set inside the nested LOO (the default, `BENCHMARK_FEATURE_SELECTION=inner`). In every outer fold, every model chooses its features using inner-LOO MSE on the training stations only. The held-out station never influences the choice.

- **Candidates.** The chemistry predictors are fixed per target. They are the only columns with missing values, so every candidate uses exactly the same rows as before. The seasonal and static parts vary: Fourier $K \in \{1, 2, 4\}$ × static ∈ {none, `local_slope`, elevation + log area + slope}. The `TARGET_CONFIGS` set is added as a further candidate ("configured"), giving 9–10 candidates per target.
- **Search.** The graph model searches feature set, weighting, λ and λ_β jointly. The other models use two stages: they choose the feature set at the middle setting of their grid, then tune their settings on that set. The dummy model ignores its inputs and is not affected.
- **Run.** One full server run, with no row thinning (`BENCHMARK_INNER_STRIDE=1`), on the same stations, rows and folds as Table 7.

Results.

- **Specific conductance.** The graph model drops by 0.05, to 0.207. This confirms that the fixed-feature 0.257 was inflated. It is still the only model clearly above zero, ahead of the next one (spline GAM, 0.022) by about 0.19.
- **pH.** The graph model is unchanged at 0.130 and still leads by about 0.11. The tree models collapse (gradient boosting −0.17, random forest −0.27). Given richer seasonal and static candidates, they choose sets that fit the training stations well but transfer poorly to a new station.

Table 9: Benchmark with inner-loop feature selection: node-weighted nested-LOO R^2. Rows, stations and folds are identical to Table 7. Bold marks the best model for each target.

Model	Water temp.	DO	Spec. cond.	pH	Turbidity
Graph-Laplacian	0.8628	0.6998	**0.2068**	**0.1299**	0.1063
OLS	0.8634	0.6951	−0.0987	−0.0138	0.1360
Ridge	**0.8640**	0.7008	−0.1096	0.0029	0.1362
Elastic Net	0.8635	0.7048	−0.1089	0.0226	0.1362
Spline GAM	0.8382	0.7076	0.0218	0.0117	0.0910
Random Forest	0.7579	0.6845	−0.1740	−0.2685	0.0996
Gradient Boosting	0.8326	**0.7327**	−0.0939	−0.1713	**0.3911**
Dummy	−0.0327	−0.0286	−0.0623	−0.0301	−0.0271
Graph rank (of 8)	4	5	1	1	5

Table 10: Change in R^2 from fixed features (Table 7) to inner-loop selection (Table 9). Negative values mean the fixed feature set had been flattering the model, or that inner selection picked worse features in some folds.

Model	Water temp.	DO	Spec. cond.	pH	Turbidity
Graph-Laplacian	0.000	−0.031	−0.050	0.000	+0.080
OLS	0.000	−0.035	+0.025	−0.025	+0.088
Ridge	0.000	−0.024	−0.010	−0.010	+0.097
Elastic Net	0.000	+0.005	+0.007	0.000	+0.100
Spline GAM	−0.011	+0.062	−0.009	+0.009	−0.011
Random Forest	+0.004	−0.009	−0.089	−0.245	−0.001
Gradient Boosting	+0.011	−0.003	−0.095	−0.175	+0.084

- **Water temperature.** Nothing material changes. The four linear-type models, including the graph model, remain within 0.0012 of each other.
- **DO.** The graph model loses 0.031 and falls from second to fifth. Its hand-picked set suited it particularly well. The top five models now fall within 0.033 of each other, which is within noise for 17 stations.
- **Turbidity.** Almost every model improves. The graph model goes from 0.026 to 0.106, the linear models from about 0.04 to 0.136, and gradient boosting from 0.307 to 0.391. The hand-picked turbidity set (Fourier $K = 1$ plus slope) was a poor choice, and the inner loop finds better ones. Gradient boosting's large lead suggests nonlinear effects that the additive models miss.

The headline result survives fair selection: the graph model wins clearly for specific conductance and pH, and for no other target.

Relation to the fair values in Table 5. For specific conductance, the benchmark's 0.207 is higher than the 0.14 from the notebook's nested selection. The candidate sets differ. The benchmark includes the "configured" set (separate daily $K = 1$ and annual $K = 4$ terms plus slope), and the graph model picks it in 22 of 24 outer folds. The notebook's 30 candidates do not include it. That set was itself designed by looking at outer-LOO results, so offering it as a candidate carries over some of the original optimism. The benchmark values are fair *between models*, since all eight models

see the same candidates. As an absolute estimate for specific conductance, 0.14 remains the more conservative figure.

7 Effect of edge weights

Every Laplacian so far has used edge weights: 1/km, or the MERIT-based schemes. Here the weights are removed ($w = 1$ on every edge) on both the original and the compressed graph. The script is `compare_edge_weights.py`. Both graphs are trees (67,401 nodes with 67,400 edges, and 67 nodes with 66 edges), so the effective resistance between two stations is just the sum of $1/w$ along the river path between them:

- **distance:** km of river (median over station pairs 844 km);
- **unweighted, original graph:** number of original segments (median 8,758);
- **unweighted, compressed graph:** number of compressed reaches (median 9, range 1–27).

A fourth Laplacian, *hops*, gives each compressed reach weight 1/(original segments in the reach), which is the unweighted original graph's segments in series.

The Laplacians. Table 11 compares the Kron matrices after the model's scaling. Since λ is tuned, only relative weights matter.

Table 11: Kron-reduced Laplacians with and without edge weights, compared after scaling. "Resistance corr." is the correlation of pairwise effective resistances between WQ stations.

Comparison	Rel. Frobenius diff.	Resistance corr.
distance: original vs compressed	5.7×10^{-13}	1.0000
unweighted: original vs compressed	0.78	0.7926
unweighted original vs hops compressed	2.1×10^{-13}	1.0000
original graph: distance vs unweighted	0.034	0.9999
compressed graph: distance vs unweighted	0.79	0.7871

- **On the original graph, weights barely matter.** The 67k segments are nearly all the same length (about 0.1 km), so counting segments is almost the same as measuring km.
- **Without weights, compression is no longer exact.** Each compressed reach counts as one step, whether it is 5 km or 245 km long. Compression stays exact only when a reach keeps the series weight of the segments it replaces. The hops Laplacian does this and matches the unweighted original graph to 10^{-13}, as the distance weights do.

Effect on R^2, chemistry predictors only. Table 12 reruns the nested LOO of Section 3 (same predictors, grids and folds) with each Laplacian. The distance columns reproduce Table 2.

On the original graph, dropping the weights changes R^2 by at most 0.002, with nearly the same λ chosen in every fold. On the compressed graph, it lowers pH, DO and turbidity by 0.01–0.02 but raises specific conductance from 0.09 to 0.24.

Selected fairly inside the benchmark. Table 12 picks the best of several Laplacians after seeing outer R^2, which is the optimism problem of Section 4.2. To test it fairly, `unweighted_compressed` was added as a sixth weighting the graph model can choose inside the inner loop of the

Table 12: Nested-LOO R^2 by edge weighting, chemistry predictors only. Hops on the compressed graph equals unweighted on the original graph to four decimals and is not shown.

Target	No graph	Distance	Unweighted, original	Unweighted, compressed
DO concentration	0.6208	0.6135	0.6131	0.5992
Water temperature	−0.0556	−0.0346	−0.0341	−0.0334
pH	−0.0352	0.0742	0.0732	0.0519
Specific conductance	−0.0891	0.0907	0.0893	0.2420
Turbidity	0.0228	−0.0486	−0.0494	−0.0633

benchmark, alongside feature set, λ and λ_β. With this weighting, λ chose the bottom of the grid (0.01) in most folds, so the grid was extended down to 10^{-4}. Table 13 gives graph-only runs with inner feature selection. The other seven models do not use the Laplacian, so their values in Table 9 are unchanged.

Table 13: Graph-Laplacian R^2 in the benchmark with inner-loop feature selection, before and after adding the unweighted compressed Laplacian as a candidate weighting, and the weighting chosen across outer folds. "Best other" is the best of the seven other models in Table 9.

Target	5 weightings	+ unweighted	Best other	Weighting chosen
Specific conductance	0.2068	0.3847	0.0218	unweighted, 24/24
pH	0.1299	0.1173	0.0226	distance, 15/19
Water temperature	0.8628	0.8634	0.8640	unweighted, 24/24
DO concentration	0.6998	0.6996	0.7327	mostly distance
Turbidity	0.1063	0.1043	0.3911	width, 14/17

- **Specific conductance.** The inner loop picks the unweighted compressed Laplacian in all 24 folds, with $\lambda = 0.03$ in 21 of them, well inside the grid. R^2 rises from 0.207 to 0.385, the same with the original or extended λ grid, so the whole gain comes from the weighting. The graph model now leads every other model by about 0.36.
- **pH.** The graph model drops by 0.013. Two folds choose the unweighted Laplacian and predict worse. With the original λ grid the value is 0.115, so the extended grid is not the cause. On 19 stations this is within noise.
- **Water temperature.** The unweighted Laplacian is chosen, but with $\lambda = 100$, very strong smoothing, so the graph term is close to a constant and R^2 barely moves.
- **DO and turbidity.** No material change.

Interpretation. For specific conductance, the number of reaches between two stations describes how they differ far better than the km of river between them. Compressed-graph nodes sit at confluences and stations, so a reach count is roughly a count of tributary junctions. That is consistent with conductance being driven by tributary inputs and mixing at confluences more than by distance downstream. It is a hypothesis this analysis cannot confirm.

8 Conclusions

1. **Kron reduction is exact for this model.** It matches the full 67,401-node Laplacian to rounding error for every weighting, and gives identical hyperparameters and R^2. It is about 150× faster, so the 26×26 Kron matrix should remain the default. Compressing the graph changes nothing either.
2. **The value of the graph term depends on the target.** It clearly helps for pH and, to a lesser extent, specific conductance. For water temperature, DO and turbidity, seasonal and MERIT predictors matter far more than the Laplacian.
3. **Report the fair numbers.** The fair R^2 values in Table 5 should replace earlier figures. In particular, specific conductance should be reported at about 0.14, not 0.22–0.27.
4. **Use the updated benchmark for model comparisons.** The graph model is now fitted inside `water_quality_model_benchmark.py` on the same data as the other models, and the full rerun (Table 7) replaces the hand-entered graph rows of the Nov 5 table, in particular turbidity ($0.0145 \rightarrow 0.026$).
5. **The graph model's advantage survives fair feature selection, but only for two targets.** With every model choosing its features inside the inner loop (Table 9), the graph model is still clearly best for specific conductance (0.207 vs at most 0.022) and pH (0.130 vs at most 0.023). It ties with the linear models for water temperature, is mid-pack for DO, and trails gradient boosting by 0.28 for turbidity. Inner-loop selection should be the default for future benchmark runs.
6. **Edge weights matter only on the compressed graph, and only for specific conductance.** On the original graph, removing weights changes almost nothing because the segments are nearly uniform in length. On the compressed graph, an unweighted Laplacian (one step per reach) is a different geometry. Chosen inside the inner loop, it raises the graph model's specific conductance R^2 from 0.207 to 0.385 and leaves the other targets within noise (Table 13).

Caveats. With only 17–24 stations, differences in R^2 below roughly 0.03–0.05 are within noise, and no standard errors were computed. For pH and specific conductance the chosen features vary between folds, another sign that those gains are fragile. The daily×annual interaction terms and separate daily and annual K from the original water-temperature section were not among the notebook's candidates. In the original they changed predictions by at most 0.009 °C. In the benchmark, the "configured" candidate was designed using outer-LOO results, so the inner-selection values in Table 9 may still be slightly optimistic for every model, most likely for specific conductance. The two-stage search chooses each non-graph model's features at a single reference setting, which may disadvantage models whose best features depend strongly on their settings. The unweighted compressed Laplacian was only tried because an exploratory run (Table 12) looked good for specific conductance, a mild second-order selection effect that the inner loop cannot remove. The gain (+0.18) is much larger than that effect plausibly explains, but it should be confirmed on another basin or a held-out time period. The Table 13 values come from local graph-only runs, not the server.

Reproducibility. All results come from `largest_uncompressed_laplacian_fixed.ipynb` and are saved in `results/laplacian_fixed/`. The standalone check script is `compare_kron_vs_full_`

laplacian.py. The benchmark comparisons in Tables 7 and 8 come from a full server run of water_quality_model_benchmark.py (commit 33e913b), saved in results/benchmark_results/. The graph rows can be rerun on their own with BENCHMARK_MODELS=Graph-Laplacian. Table 9 comes from a full server run of the same script at commit b72b9b4, with its defaults (inner selection, two-stage search, stride 1). Its results, including feature_selection_counts.csv with the feature set chosen in each outer fold, are written to results/benchmark_results_inner_fs/. To reproduce the fixed-feature tables, set BENCHMARK_FEATURE_SELECTION=fixed. Tables 11 and 12 come from compare_edge_weights.py (about a minute), saved in results/edge_weight_matrix_checks.csv and results/edge_weight_r2.csv. Table 13 uses the benchmark at commit c1b3dde with BENCHMARK_MODELS=Graph-Laplacian. Setting BENCHMARK_GRAPH_WEIGHTINGS=distance_only,area,width,slope,area_width and BENCHMARK_GRAPH_LAMBDA_GRID=0.01,0.03,0.1,0.3,1,3,10,30,100 restores the earlier graph model.